Unit 1

php introduction

1. PHP is a server-side scripting language created primarily for web development but it is also used as a general-purpose programming language.
2. Unlike client-side languages like JavaScript, which are executed on the user's browser, PHP scripts run on the server.
3. The results are then sent to the client's web browser as plain HTML.
4. It can be integrated with many databases such as Oracle, Microsoft SQL Server, MySQL, PostgreSQL, Sybase, and Informix.
5. It is powerful to hold a content management system like WordPress and can be used to control user access.
6. PHP is dynamically typed, meaning you don't need to declare the data type of a variable explicitly.
7. Websites like www.facebook.com/ and www.yahoo.com/ are also built on PHP.

Advantages of php

- **Easy to learn**: PHP has a simple syntax that's similar to HTML, making it easy for beginners to learn.
- **Cost-effective**: PHP is free and open-source, so there's no need to buy licenses or software.
- **Scalable**: PHP can handle increased traffic and user interactions by using caching, load balancing, and code optimization.
- **Platform-independent**: PHP runs on many operating systems, including Windows, Linux, and Unix.
- **Database-friendly**: PHP can connect to databases quickly and securely.
- **Extensive library**: PHP has a large library of pre-written code, so you don't need to start from scratch.
- **Cloud-compatible**: PHP-based applications can run on cloud computing platforms like Amazon Web Services.
- **Integratable**: PHP can integrate with other programming languages and technologies, like Java.
- **Flexible**: PHP can be used with many different operating systems and programming languages.
- **Gives developers more control**: PHP gives developers more control than other programming languages.

The use of $ in php/variables in php

1. All variable names start with a dollar sign ($). This tells PHP that it is a variable name.
2. Variable names can be any length.
3. Variable names can include letters, numbers, and underscores only.
4. Variable names must begin with a letter or an underscore.
5. They cannot begin with a number.
6. Variable names in PHP are case-sensitive, so "$name" is different from "$Name".

---

constants

1. Constants in PHP are identifiers or names that can be assigned any fixed values and cannot change during the execution of a program.
2. They remain constant throughout the program and cannot be changed during the execution.
3. Once a constant is defined, it cannot be undefined or redefined.
4. By convension, constant identifiers are always written in upper case.
5. By default, a constant is always case-sensitive.
6. A constant name must never start with a number.
7. It always starts with a letter or underscores, followed by letter, numbers or underscore.
8. It should not contain any special characters except underscore, as mentioned.

Data types

1. **Integer:** Whole numbers, positive or negative, without any decimal point.
```
5, -23, 100
```
2. **Float (or Double):** Numbers with decimal points or numbers in exponential form.
```
3.14, -0.5, 2.1e3
```
3. **String:** A sequence of characters enclosed in single or double quotes.
```
"Hello World", 'PHP'
```
4. **Boolean:** Represents two possible states: TRUE or FALSE.
```
TRUE, FALSE
```
5. **Array:** A collection of values that can be of different data types, stored under a single variable name.

- Indexed Array: An array with numeric indexes.

```
$arr = [1, 2, 3]
```

- Associative Array: An array where each element is associated with a key (string or integer).

```
$arr = ['name' => 'John', 'age' => 30]
```
6. **NULL:** Represents a variable with no value or an empty state.
```
$var = NULL;
```
7. **Resource:** A special variable used to store references to external resources like database connections or file handles.
```
$file = fopen("file.txt", "r");
```

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y
**	Exponentiation	$x ** $y	Result of raising $x to the $y'th power (Introduced in PHP 5.6)

Assignment operator

Assignment	Same as...	Description
x = y	x = y	The left operand gets set to the value of the expression on the right
x += y	x = x + y	Addition
x -= y	x = x – y	Subtraction
x *= y	x = x * y	Multiplication
x /= y	x = x / y	Division
x %= y	x = x % y	Modulus

Comparison operator

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y

---

Increment/Decrement operator

The PHP decrement operators are used to decrement a variable's value.

Operator	Name	Description
++$x	Pre-increment	Increments $x by one, then returns $x
$x++	Post-increment	Returns $x, then increments $x by one
--$x	Pre-decrement	Decrements $x by one, then returns $x
$x--	Post-decrement	Returns $x, then decrements $x by one

Logical operators

Operator	Name	Example	Result
And	And	$x and $y	True if both $x and $y are true
Or	Or	$x or $y	True if either $x or $y is true
Xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

String operators

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

Array operators

Operator	Name	Example	Result
+	Union	$x + $y	Union of $x and $y
==	Equality	$x == $y	Returns true if $x and $y have the same key/value pairs
===	Identity	$x === $y	Returns true if $x and $y have the same key/value pairs in the same order and of the same types
!=	Inequality	$x != $y	Returns true if $x is not equal to $y
<>	Inequality	$x <> $y	Returns true if $x is not equal to $y
!==	Non-identity	$x !== $y	Returns true if $x is not identical to $y

---

Conditional statements

1. **`if` Statement:** The `if` statement checks a condition, and if the condition is true, it executes the block of code inside the statement.

```
$age = 20;
if ($age >= 18) {
    echo "You are eligible to vote.";
}
```

2. **`if...else` Statement:** The `if...else` statement allows you to define a block of code to be executed if the condition is true, and another block of code to be executed if the condition is false.

```
$age = 15;
if ($age >= 18) {
    echo "You are eligible to vote.";
} else {
    echo "You are not eligible to vote.";
}
```

3. **`if...elseif...else` Statement:** The `if...elseif...else` statement checks multiple conditions. It allows you to specify different blocks of code to execute for different conditions.

```
$age = 30;
if ($age < 18) {
    echo "You are a minor.";
} elseif ($age >= 18 && $age <= 60) {
    echo "You are an adult.";
} else {
    echo "You are a senior citizen.";
}
```

4. **`switch` Statement:** The `switch` statement is used to perform different actions based on different conditions. It's useful when you have many conditions to check for a single variable.

```
$day = 3;
switch ($day) {
    case 1:
        echo "Monday";
        break;
    case 2:
        echo "Tuesday";
        break;
    case 3:
        echo "Wednesday";
        break;
    case 4:
        echo "Thursday";
        break;
    case 5:
        echo "Friday";
        break;
    case 6:
        echo "Saturday";
        break;
    case 7:
        echo "Sunday";
```

```
        break;
    default:
        echo "Invalid day";
}
```

5. **Ternary Operator:** The ternary operator is a shorthand for an `if...else` statement. It is used to return one of two values depending on the condition.

```
$age = 20;
echo ($age >= 18) ? "You are eligible to vote." : "You are not eligible to
vote.";
```

6. **Null Coalescing Operator (`??`):** The null coalescing operator returns the value of its left-hand operand if it exists and is not null; otherwise, it returns the value of the right-hand operand.

```
$username = $_GET['user'] ?? 'Guest';
echo $username; // If 'user' is not set in the query string, it will print
'Guest'.
```

Comments

```
# This is a single line commment
//This/ is also a single line comment

/*This is a
multiline comment*/
```

PHP string functions

Function	Description	Example	Output
Strtolower	Used to convert all string characters to lower case letters	echo strtolower('Benjamin');	outputs benjamin
Strtoupper	Used to convert all string characters to upper case letters	echo strtoupper('george w bush');	outputs GEORGE W BUSH
Strlen	The string length function is used to count the number of character in a string. Spaces in between characters are also counted	echo strlen('united states of america');	24
Explode	Used to convert strings into an array variable	$settings = explode(';', "host=localhost; db=sales; uid=root; pwd=demo"); print_r($settings);	Array ([0] => host=localhost [1] =>db=sales [2] =>uid=root [3] =>pwd=demo)
Substr	Used to return part of the string. It accepts three (3) basic parameters. The first one is the string to be shortened, the second parameter is the position of the starting point, and the third parameter is the number of characters to be returned.	$my_var = 'This is a really long sentence that I wish to cut short';echosubstr($my_var,0, 12).'...';	This is a re...
str_replace	Used to locate and replace specified string values in a given string. The function accepts three arguments. The first argument is the text to be replaced, the second argument is	echo str_replace ('the', 'that', 'the laptop is very expensive');	that laptop is very expensive

Function	Description	Example	Output
	the replacement text and the third argument is the text that is analyzed.		
Strops	Used to locate the and return the position of a character(s) within a string. This function accepts two arguments	echo strpos('PHP Programing','Pro');	4
sha1	Used to calculate the SHA-1 hash of a string value	echo sha1('password');	5baa61e4c 9b93f3f0 682250b6cf8331b 7ee68fd8
md5	Used to calculate the md5 hash of a string value	echo md5('password');	9f961034ee 4de758 baf4de09ceeb1a75
str_word_count	Used to count the number of words in a string.	echo str_word_count ('This is a really long sentence that I wish to cut short');	12
Ucfirst	Make the first character of a string value upper case	echo ucfirst('respect');	Outputs Respect
Lcfirst	Make the first character of a string value lower case	echo lcfirst('RESPECT');	Outputs rESPECT

Numeric Functions

Function	Description	Example	Output
		} ?>	
number_format	Used to formats a numeric value using digit separators and decimal points	`number_format(2509663);`	2,509,663
Rand	Used to generate a random number.	`echo rand();`	Random number
Round	Round off a number with decimal points to the nearest whole number.	`echo round(3.49);`	3
Sqrt	Returns the square root of a number	`echo sqrt(100);`	10
Cos	Returns the cosine	`echo cos(45);`	0.52532198881773
Sin	Returns the sine	`echo sin(45);`	0.85090352453412
Tan	Returns the tangent	`echo tan(45);`	1.6197751905439
Pi	Constant that returns the value of PI	`echo pi();`	3.1415926535898

Getting the Current Date and Tirme

1. The `date()` function formats a local date and time based on a specified format.

```
date(format, timestamp);
```

2. The `time()` function returns the current Unix timestamp (number of seconds since January 1, 1970, 00:00:00 UTC).

```
echo time(); // Outputs: 1678114416 (Unix timestamp)
```

Date and Time Format Characters

1. Day Formats:

- `d` – Day of the month (01 to 31)

- `D` – A three-letter abbreviation for the day of the week (Mon, Tue, etc.)
- `j` – Day of the month without leading zeros (1 to 31)
- `l` – Full name of the day of the week (Monday, Tuesday, etc.)
- `N` – ISO-8601 numeric representation of the day of the week (1 for Monday, 7 for Sunday)
- `S` – English ordinal suffix for the day of the month (st, nd, rd, th)

2. Month Formats:

- `m` – Numeric representation of the month (01 to 12)
- `M` – A three-letter abbreviation for the month (Jan, Feb, etc.)
- `n` – Numeric representation of the month without leading zeros (1 to 12)
- `F` – Full name of the month (January, February, etc.)
- `t` – Number of days in the given month (28 to 31)

3. Year Formats:

- `Y` – Four-digit representation of the year (2025)
- `y` – Two-digit representation of the year (25)

4. Time Formats:

- `H` – 24-hour format of an hour (00 to 23)
- `i` – Minutes with leading zeros (00 to 59)
- `s` – Seconds with leading zeros (00 to 59)
- `a` – Lowercase am or pm
- `A` – Uppercase AM or PM
- `g` – 12-hour format of an hour without leading zeros (1 to 12)
- `G` – 24-hour format of an hour without leading zeros (0 to 23)

Unit 2

Indexed arrays

An indexed array is an array where the elements are accessed using numeric indices (starting from 0 by default).
There are two ways to define an indexed array:

1. Indexed Array with Numeric Index

```
$fruits = array("Apple", "Banana", "Cherry");
echo $fruits[0]; // Outputs: Apple
```

2. Indexed Array with Custom Numeric Index

```
$fruits = array(0 => "Apple", 1 => "Banana", 2 => "Cherry");
echo $fruits[1]; // Outputs: Banana
```

3. Using the `[]` shorthand to create an indexed array

```
$fruits = ["Apple", "Banana", "Cherry"];
echo $fruits[2]; // Outputs: Cherry
```

Multidimensional Arrays in PHP

A **multidimensional array** is an array that contains one or more arrays. You can think of it like a matrix or a table of rows and columns.

```
$students = array(
    array("Alice", 20, "A"),
    array("Bob", 22, "B"),
    array("Charlie", 21, "A")
);

echo $students[1][0]; // Outputs: Bob
```

Accessing and Manipulating Arrays in PHP

1. Accessing elements: You can access array elements using the index or key (for associative arrays).

```
echo $fruits[0]; // For indexed arrays
echo $grades["Math"]["Bob"]; // For associative arrays
```

2. **Modifying elements:** You can modify the value of an array element by assigning a new value to a specific index or key.

```
$fruits[1] = "Mango"; // Changes "Banana" to "Mango"
$grades["Science"]["Alice"] = 95; // Changes Alice's grade in Science
```

3. **Looping through arrays:**Use `foreach` to loop through indexed and associative arrays.

```
// Indexed array
foreach($fruits as $fruit) {
    echo $fruit . "<br>";
}

// Associative array
foreach($grades as $subject => $students) {
    echo $subject . ":<br>";
    foreach($students as $student => $grade) {
        echo "$student - $grade<br>";
    }
}
```

---

built-in array sorting functions

1. sort(): This function sorts the array in ascending order and reindexes the array (the keys are reset to 0, 1, 2, ...).

```
$fruits = ["Banana", "Apple", "Cherry"];
sort($fruits);
print_r($fruits);
// Output: Array ( [0] => Apple [1] => Banana [2] => Cherry )
```

2. rsort(): This function sorts the array in descending order and reindexes the array.

```
$fruits = ["Banana", "Apple", "Cherry"];
rsort($fruits);
print_r($fruits);
// Output: Array ( [0] => Cherry [1] => Banana [2] => Apple )
```

3. asort(): This function sorts the array in ascending order according to the values while maintaining the keys.

```
$fruits = ["a" => "Banana", "b" => "Apple", "c" => "Cherry"];
asort($fruits);
print_r($fruits);
// Output: Array ( [b] => Apple [a] => Banana [c] => Cherry )
```

4. ksort(): This function sorts the array by the keys in ascending order while maintaining the key-value pairs.

```
$fruits = ["a" => "Banana", "c" => "Apple", "b" => "Cherry"];
ksort($fruits);
print_r($fruits);
// Output: Array ( [a] => Banana [b] => Cherry [c] => Apple )
```

5. arsort(): This function sorts the array in descending order according to the values while maintaining the keys.

```
$fruits = ["a" => "Banana", "b" => "Apple", "c" => "Cherry"];
arsort($fruits);
print_r($fruits);
// Output: Array ( [c] => Cherry [a] => Banana [b] => Apple )
```

6. ksort(): This function sorts the array by the keys in ascending order while maintaining the key-value pairs.

```
$fruits = ["a" => "Banana", "c" => "Apple", "b" => "Cherry"];
ksort($fruits);
print_r($fruits);
// Output: Array ( [a] => Banana [b] => Cherry [c] => Apple )
```

Request Method

1. The $_REQUEST method in PHP is a superglobal array that is used to collect data after submitting forms.
2. It can contain data sent via GET, POST, and COOKIE methods, making it a convenient way to access information from various sources in a single array.
3. It combines the values from $_GET, $_POST, and $_COOKIE.
4. It's useful when you don't know exactly whether data will be sent using GET or POST, or when both GET and POST might be involved.
5. The $_REQUEST array will give preference to POST data, then GET, and finally COOKIE data if they share the same key.
6.
```
<form method="get" action="example.php">
    <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```
7.
```
example.php
echo $_REQUEST['name'];  // Outputs the value of the 'name' parameter
```

GET method

1. The GET method is used in web development to request data from a server.
2. It is one of the most common HTTP request methods.
3. When you enter a website URL or click a link, your browser sends a GET request to the server, asking for the webpage.
4. It Retrieves data like a webpage, image, or API response.
5. Data is visible in the URL (e.g., example.com/search?q=apple).
6. It Does not modify server data, only fetches information.
7. It is Cached by browsers to improve speed for repeated requests.
8. It has Limited data length because URLs have size limits.

POST method

1. The POST method is used in web development to send data to a server to create or update resources.
2. Unlike the GET method, POST sends data in the request body, making it more secure and suitable for sensitive information like passwords or form data.
3. Sends data to the server (e.g., submitting a form, uploading a file).
4. Data is not visible in the URL instead it is sent in the request body).
5. Modifies server data like creating a new user or post).
6. It is not cached by browsers it ensures fresh requests.
7. There is no data size limit unlike GET.

Difference in GET and POST

Parameter	GET	POST
Visibility	Data is appended to the URL and visible in the browser's address bar.	Data is included in the request body and not visible in the URL.
Data Length	Limited by the URL length, which can vary by browser and server. Typically, around 2048 characters.	No inherent limit on data size, allowing for large amounts of data to be sent.
Security	Less secure due to visibility in the URL. Sensitive data can be easily exposed in browser history, server logs, etc.	More secure for transmitting sensitive data since it's not exposed in the URL.
Use Case	Ideal for simple data retrieval where the data can be bookmarked or shared.	Suited for transactions that result in a change on the server, such as updating data, submitting forms, and uploading files.
Idempotency	Idempotent, meaning multiple identical requests have the same effect as a single request.	Non-idempotent, meaning multiple identical requests, may have different outcomes.
Caching	Can be cached by the browser and proxies.	Not cached by default since it can change the server state.
Data Type	Only ASCII characters are allowed. Non-ASCII characters must be encoded.	Can handle binary data in addition to text, making it suitable for file uploads.
Back/Forward Buttons	Reloading a page requested by GET does not usually require browser confirmation.	Reloading a page can cause the browser to prompt the user for confirmation to resubmit the POST request.
Bookmarks and Sharing	Easily bookmarked and shared since the data is part of the URL.	Cannot be bookmarked or shared through the URL since the data is in the request body.
Impact on Server	Generally used for retrieving data without any side effects on the server.	Often causes a change in server state (e.g., database updates) or side effects.

PHP validation

PHP validation ensures that user input is correct, secure, and follows required formats before processing. It is divided into **client-side** (JavaScript) and **server-side** (PHP) validation, but **server-side validation is essential** for security.

Types of PHP Validation:

1. **Form Validation** – Checks if required fields are filled.
2. **Data Type Validation** – Ensures correct input type (e.g., numbers for age).
3. **Format Validation** – Uses regex for emails, phone numbers, etc.
4. **Length Validation** – Limits input size to prevent overflow.
5. **Security Validation** – Prevents SQL Injection, XSS attacks.

Unit 3

File Handling in PHP

File handling in PHP allows us to create, read, write, and manage files on a server. It is useful for storing logs, user data, or configurations.

1. Opening a File

- PHP uses the `fopen()` function to open a file.
- Syntax:

```
$file = fopen("example.txt", "r"); // Opens file in read mode
```

- Modes:
  - `"r"` – Read only
  - `"w"` – Write only (erases file if it exists)
  - `"a"` – Append mode (writes at the end of the file)
  - `"x"` – Create new file (fails if file exists)

2. Reading a File

- The `fread()` function reads content from a file.
- Example:

```
$file = fopen("example.txt", "r");
echo fread($file, filesize("example.txt"));
fclose($file);
```

3. Writing to a File

- The `fwrite()` function writes content to a file.
- Example:

```
$file = fopen("example.txt", "w");
fwrite($file, "Hello, PHP File Handling!");
fclose($file);
```

4. Closing a File

```
fclose($file); to free system resources.
```

5. Checking if a File Exists

```
file_exists("example.txt") checks if a file is present.
```

6. Deleting a File

```
unlink("example.txt"); deletes the file.
```

Session

1. A session in PHP is used to store user information across multiple web pages.
2. Unlike cookies, which store data in the user's browser, session data is stored on the server, making it more secure.
3. A session starts when a user visits a website, and PHP assigns a unique session ID to track the user.
4. This session ID is stored in the browser as a cookie, allowing PHP to identify the user's session across different pages.
5. We can store data like usernames, preferences, or shopping cart items in a session.
6. Session variables remain available throughout the user's visit until they leave or the session expires.
7. By default, a session lasts until the user closes the browser.
8. The session timeout can be adjusted based on the website's requirements to automatically log out inactive users.

Cookie

1. A cookie is a small piece of data stored in the user's browser.
2. It helps websites remember user preferences, login details, or other information between visits.
3. When a website sets a cookie, the data is saved in the user's browser.
4. On future visits, the browser sends the cookie back to the server.
5. This allows websites to remember user preferences, login sessions, or personalized settings.
6. A cookie is created with a name, value, and optional expiration time.
7. Once set, a cookie can be accessed on any page of the website.
8. The stored value can be retrieved and used to personalize the user experience.
9. By default, a cookie expires when the user closes the browser.
10. Once expired, the browser removes the cookie, and it is no longer accessible.

Difference Between Session and Cookie

Feature	Session	Cookie
Storage Location	Stored on the server	Stored in the user's browser
Security	More secure (data is not exposed to users)	Less secure (stored in the browser and can be accessed by users)
Data Size	Can store large amounts of data	Limited storage (usually 4KB per cookie)
Lifetime	Ends when the user closes the browser (unless configured otherwise)	Can persist even after the browser is closed (if an expiry time is set)
Speed	Slightly slower as it requires server processing	Faster as data is stored locally
Usage	Used for login sessions, shopping carts, and storing sensitive data	Used for remembering user preferences, tracking, and analytics
Dependency on Browser	Works regardless of browser settings	May not work if the user disables cookies in their browser
Accessibility	Accessible only on the server	Accessible both by the server and the user's browser

---

Filters in PHP

Filters in PHP are used to **validate** and **sanitize** input data, ensuring security and correctness. They help prevent security risks like **SQL injection** and **XSS attacks**.

Types of Filters

1. Validating Data

- Used to check if input is in the correct format, such as email or integer.
- Example checks:
  - Is the value an integer?
  - Is it a valid email format?
  - Is it a valid URL?

2. Sanitizing Data

- Used to remove unwanted characters from input.
- Example sanitization:
  - Removing special characters from a string.
  - Removing HTML tags from user input.
  - Filtering out non-numeric characters from numbers.

3. Common PHP Filters

- `FILTER_VALIDATE_INT` – Checks if the input is a valid integer.
- `FILTER_VALIDATE_EMAIL` – Checks if input is a valid email.
- `FILTER_VALIDATE_URL` – Checks if input is a valid URL.
- `FILTER_SANITIZE_STRING` – Removes unwanted characters from a string.
- `FILTER_SANITIZE_EMAIL` – Removes illegal characters from an email.

Unit 4

Error Handling in PHP

Error handling in PHP ensures that the script runs smoothly and provides meaningful error messages when something goes wrong. PHP provides several methods to handle errors like:

1. Using `die()` Function

- The `die()` function is used to **stop script execution** if an error occurs.
- It can display an error message before terminating the script.
- Commonly used in file handling and database connections.
- Example: If a file is missing, `die()` will stop execution instead of continuing with an error.

2. Using `set_error_handler()`

- The `set_error_handler()` function allows you to **define a custom error handler**.
- Instead of PHP displaying default error messages, you can create your own error messages.
- Useful for logging errors or displaying user-friendly error messages.

3. Using `trigger_error()`

- The `trigger_error()` function is used to **generate a custom error message**.
- It allows developers to manually trigger errors with a custom message.
- You can specify error types such as:
  - `E_USER_ERROR` (Fatal error)
  - `E_USER_WARNING` (Warning)
  - `E_USER_NOTICE` (Informational message)
- Useful for debugging or stopping execution when incorrect data is encountered.

Types of Errors in PHP

1. Fatal Error (`E_ERROR`)

- Occurs when a script encounters a serious issue and **cannot continue execution**.
- Example causes:
  - Calling an undefined function.
  - Using a class that does not exist.
- The script **stops immediately** when a fatal error occurs.

2. Warning (`E_WARNING`)

- A **non-fatal error** that **does not stop** script execution.
- Example causes:
  - Including a missing file.
  - Performing invalid operations like dividing by zero.
- The script continues running even after the warning is displayed.

3. Notice (`E_NOTICE`)

- A minor error that **does not stop** execution.
- Example causes:
  - Using an undefined variable.
  - Accessing an array index that does not exist.

4. Parse Error (`E_PARSE`)

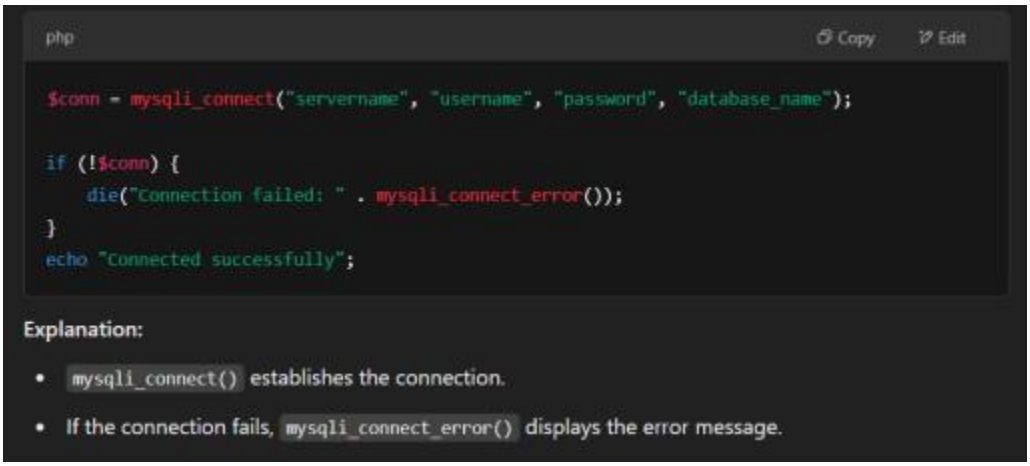
- Also known as a **syntax error**, occurs when PHP **fails to interpret** the script due to incorrect syntax.
- Example causes:
  - Missing semicolons (`;`).
  - Using incorrect brackets `{}` or `()`.
- This error **prevents the script from running** until fixed.

Keywords in PHP

Keyword	Description
`function`	Defines a reusable block of code. Used to perform a specific task.
`class`	Declares a class in object-oriented programming, allowing object creation.
`extends`	Enables inheritance, allowing a class to inherit properties and methods from another class.
`include`	Includes an external PHP file. If the file is missing, it shows a warning but continues execution.
`require`	Similar to `include`, but if the file is missing, it stops script execution with a fatal error.
`echo`	Displays output on the webpage. It is faster than `print` and does not return a value.
`isset`	Checks if a variable is set and is not `NULL`. Returns `true` if the variable exists.
`unset`	Destroys a variable, making it undefined.
`try-catch`	Handles exceptions in PHP by catching and managing errors instead of stopping execution.
`static`	Preserves a variable's value across multiple function calls instead of resetting it.

Unit 5

syntax of connecting php webpage with mysql



```php
$conn = mysqli_connect("servername", "username", "password", "database_name");

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
```

Explanation:

- `mysqli_connect()` establishes the connection.
- If the connection fails, `mysqli_connect_error()` displays the error message.

Prepared Statement

1. A Prepared Statement in MySQL is a feature that allows you to execute the same SQL query multiple times with different values efficiently.
2. It helps in improving performance and security, especially when dealing with user inputs.
3. Since the query structure is fixed, malicious inputs cannot alter the query.
4. MySQL compiles the query once and executes it multiple times with different values.
5. The query is parsed only once, reducing database load.
6. You can bind different values at runtime without modifying the query.
7. The separation of query structure and values makes code cleaner.

Methods to Connect MySQL Database in PHP

There are three main ways to connect a MySQL database in PHP:

1. **MySQLi Procedural Method**
   - Uses functions to establish a connection.
   - Simple and easy to implement.
   - Suitable for small applications.
2. **MySQLi Object-Oriented Method**
   - Uses an object-oriented approach with the `mysqli` class.
   - More structured and reusable.
   - Better for larger applications.
3. **PDO (PHP Data Objects) Method**
   - Supports multiple databases like MySQL, PostgreSQL, and SQLite.
   - More secure and flexible.
   - Uses exception handling for better error management.

MySQLi Query in PHP

A **MySQLi query** is used to execute SQL commands in a MySQL database using PHP. It allows interaction with the database, such as retrieving, inserting, updating, and deleting records.

Types of MySQLi Queries

1. **SELECT Query** – Retrieves data from the database.
2. **INSERT Query** – Adds new records to a table.
3. **UPDATE Query** – Modifies existing records.
4. **DELETE Query** – Removes records from a table.
5. **Other Queries** – Includes table creation, dropping, and altering queries.

Unit 6

Object-Oriented Programming (OOP) in PHP

OOP in PHP is a programming approach where code is structured into objects that contain both **data (properties)** and **functions (methods)**. It makes the code more organized, reusable, and scalable.

Key Concepts of OOP in PHP

1. **Class**: A blueprint for creating objects.
2. **Object**: An instance of a class.
3. **Properties**: Variables inside a class (also called attributes).
4. **Methods**: Functions inside a class that define behavior.
5. **Constructor (`__construct`)**: A special method that runs automatically when an object is created.
6. **Inheritance**: A class can inherit properties and methods from another class.
7. **Encapsulation**: Restricting direct access to class properties (using `public`, `private`, `protected`).
8. **Polymorphism**: Overriding or modifying methods in child classes.
9. **Abstraction**: Hiding implementation details and showing only essential features.

Constructor and Destructor

1. Constructor (`__construct()`)

- A **constructor** is a special function in a class that runs **automatically** when an object is created.
- It is used to **initialize object properties** when an instance of the class is made.
- The constructor function in PHP is always named `__construct()`.
- It helps in **reducing redundant code** by assigning initial values directly when the object is created.
- It does not need to be called manually.
- It can accept parameters to set property values at the time of object creation.
- Useful in scenarios like setting default values or opening database connections.

2. Destructor (`__destruct()`)

- A **destructor** is a special function that runs **automatically** when an object is **destroyed** or when the script execution ends.
- It is used for **cleaning up resources**, such as closing database connections, releasing memory, or deleting temporary files.
- The destructor function in PHP is named `__destruct()`.
- It does not accept parameters.
- It is executed when the object is no longer needed or at the end of the script.
- Helps in **resource management and preventing memory leaks**.

Differences Between Constructor and Destructor

Feature	Constructor (`__construct`)	Destructor (`__destruct`)
Purpose	Initializes object properties	Cleans up resources
When it Runs	When an object is created	When an object is destroyed or script ends
Accepts Parameters?	Yes	No
Manual Calling Needed?	No, runs automatically	No, runs automatically

---

Access Modifiers

In PHP, **visibility** (or **access modifiers**) defines **which parts of the program can access properties and methods** of a class. It is used for **encapsulation**, helping to control access and protect data.
There are **three types of visibility in PHP**:

1. Public

- Members (properties or methods) declared as `public` can be accessed **from anywhere**.
- That means both inside and outside of the class.

2. Protected

- Members declared as `protected` can be accessed **only within the class itself** and **by classes that inherit from it** (subclasses).
- Cannot be accessed from outside the class.

3. Private

- Members declared as `private` can be accessed **only within the class** where they are defined.
- Even subclasses cannot access private members.

Modifier	Accessible from Same Class	Accessible from Subclass	Accessible from Outside
Public	✅	✅	✅
Protected	✅	✅	❌
Private	✅	❌	❌

These modifiers help in creating **secure, modular, and well-structured code**.

Interface in PHP

1. An interface in PHP is like a contract or blueprint for classes.
2. It defines a set of methods that any class must implement if it agrees to use that interface.
3. Only method declarations – No actual code inside methods.
4. A class that uses an interface must implement all its methods.
5. Helps in achieving abstraction (hiding internal details).
6. A class can implement multiple interfaces (PHP allows multiple interfaces).
7. Interface methods are always public by default.

📦 Example in Real Life:
Think of an interface like a **remote control** – different brands of TVs may implement it differently, but the buttons (methods) remain the same.

---

final Keyword in PHP

1. The **`final` keyword** in PHP is used to **prevent inheritance or method overriding**.
2. If a class is declared as `final`, **no other class can extend it**.
3. Used when you want to **stop further inheritance**.
4. If a method is declared as `final`, **it cannot be overridden** in a subclass.
5. Used to **protect important behavior** in base classes from being changed.
6. To **secure critical code** that should not be altered by subclasses.
7. To enforce **strict rules** in object-oriented design.
8. To avoid **unexpected behaviors** from overridden methods.

Unit 7

PHP Framework

A **PHP framework** is a structured platform with **pre-built tools, libraries, and rules** that help developers **build web applications faster, securely, and in an organized way**.

1. **Faster Development:**
   - Reusable code, built-in functions, and tools speed up development.
2. **Code Organization:**
   - Most frameworks follow the **MVC (Model-View-Controller)** pattern which separates logic, design, and data clearly.
3. **Security:**
   - Built-in protection against common attacks like **SQL injection, CSRF, and XSS**.
4. **Maintainability:**
   - Structured and readable code makes it easier to **maintain or update** projects later.
5. **Scalability:**
   - Ideal for growing applications with high traffic or complex features.
6. **Built-in Tools:**
   - Includes **routing, form validation, session handling, database abstraction**, etc.
7. **Community Support:**
   - Popular frameworks have large communities, documentation, and tutorials.

Laravel

1. **Most popular PHP framework** with modern tools.
2. **MVC architecture** for clean separation of logic and design.
3. Uses **Blade templating engine** for dynamic HTML generation.
4. **Eloquent ORM** for easy database interaction using PHP syntax.
5. **Artisan CLI** for managing tasks like migration, testing, and routing via command line.
6. Built-in features: **authentication, email, queues, file storage**, and more.
7. Supports **RESTful routing** and **middleware** for HTTP request filtering.
8. Easily integrates with **Vue.js, React, Livewire** for modern UIs.
9. Huge community and **rich documentation**.
10. Best for **modern, scalable web apps** with complex requirements.

CodeIgniter

1. Lightweight and **blazing fast**.
2. **Minimal configuration**, easy to install and run.
3. Suitable for **shared hosting** and smaller servers.
4. Good for **simple or medium-level web apps**.
5. Supports **MVC**, but not strictly enforced (you can skip it).
6. Has basic tools like **form validation, session management, file uploads**.
7. Easy to learn – perfect for **beginners**.
8. Lacks modern tools like built-in authentication, but can be added manually.
9. Less opinionated – gives more **freedom** to the developer.

CakePHP

1. Uses **convention over configuration** – you just follow naming rules.
2. Strong **CRUD support** – easy to create, read, update, delete operations.
3. Built-in tools for **validation, authentication, ACL (Access Control Lists)**.
4. Follows **MVC** structure strictly.
5. Uses **Cake ORM** for database access.
6. Integrated **bake tool** (code generator) helps speed up development.
7. Secure – protects against **SQL injection, CSRF, XSS** by default.
8. Good for developers who want to write **less code with more structure**.

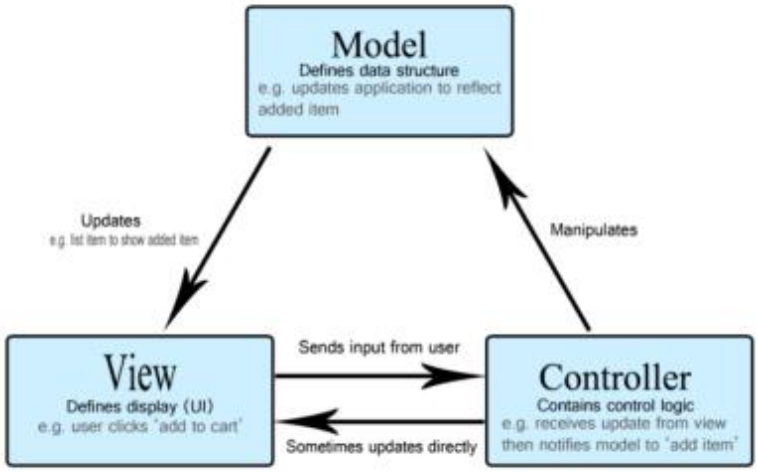
Yii (Yes, It Is)

1. Known for **performance and speed**.
2. Comes with **Gii** – a powerful code generator for models, controllers, etc.
3. Fully **object-oriented**, built using modern PHP standards.
4. Follows **MVC** and supports **RESTful API** development.
5. Integrated with **RBAC (Role-Based Access Control)** for advanced security.
6. Uses **Yii ActiveRecord ORM** for database interaction.
7. Good for **rapid development** – especially CRUD-based systems.
8. Secure, with **built-in authentication, input validation, and caching**.
9. Suitable for **medium to large applications**.

Difference between the frameworks

Feature	Laravel	CodeIgniter	CakePHP	Yii
Learning Curve	Medium	Easy	Medium	Medium
Performance	Moderate	Fast	Moderate	Very Fast
MVC Support	Strict	Optional	Strict	Strict
CLI Tool	Artisan	Basic	Bake	Gii
Templating Engine	Blade	PHP	PHP	PHP
Database ORM	Eloquent	Manual	Cake ORM	ActiveRecord
Best For	Modern apps	Beginners	CRUD apps	Enterprise

MVC Architecture



The **MVC architecture** is a software design pattern that divides an application into **three interconnected components**: Model, View, and Controller. This separation helps manage complexity and allows for efficient code organization.

Model

- Central component of the architecture.
- Represents the **dynamic data structure** of the application.
- Independent of the user interface (UI).
- **Manages the application's data, logic, and rules**.
- **Receives data from the controller** and processes it.

View

- Represents the **presentation layer** of the application.
- Displays data to the user, e.g., **tables, charts, diagrams, maps**, etc.
- Can show **multiple views of the same data** (e.g., chart for management, table for accountants).
- **Renders data** provided by the model into a **user-friendly format**.

Controller

- Acts as an **intermediary between Model and View**.
- **Handles user input** (mouse clicks, form submissions, etc.).
- **Validates and processes** the input.
- Transforms input into **model updates** or **view updates**.
- Ensures the right data flows to the model and the right view is displayed.
-

Goals of MVC (Model-View-Controller)

The **MVC architecture** is designed to **separate concerns** in web application development. Here's a clear, point-wise breakdown of its main goals:

1. Separation of Concerns

- Divides the application into 3 layers:
  - **Model** → handles data and business logic
  - **View** → handles UI (what the user sees)
  - **Controller** → manages user input and communication between model & view

2. Better Code Organization

- Makes code modular, readable, and maintainable.
- Each component has a clear role, avoiding messy code mixing.

3. Reusability

- Views and models can be reused in different parts of the application.

4. Easier Maintenance and Updates

- Since logic and UI are separate, changes in one layer don't affect the others.

5. Scalability

- Encourages building applications that are easier to scale with new features.

6. Parallel Development

Multiple developers can work simultaneously:

- One on the UI (View)
- One on the logic (Model)
- One on the control flow (Controller)

7. Improves Testing

- Logic is isolated from presentation, making **unit testing** and **debugging** easier.

MVC Advantages and Disadvantages

MVC Architecture – Advantages vs Disadvantages

Advantages	Disadvantages
1. **Separation of concerns** makes code more organized and manageable.	1. Can be **overkill for small applications** with simple logic.
2. **Supports parallel development** – different teams can work on Model, View, Controller separately.	2. **Complex to learn** for beginners, especially understanding interactions.
3. Easier to **maintain, update, and scale** the application.	3. Requires more **files and code structure**, increasing development time.
4. Improves **testability** – logic is separated from UI, making unit testing easier.	4. **Overhead of designing MVC** correctly in the initial stages.
5. Allows for **multiple views** of the same data (e.g., mobile, desktop).	5. Tight coupling between controller and view can occur if not implemented well.
6. Enhances **code reusability** due to modular components.	6. Beginners might struggle with **MVC flow and lifecycle** understanding.

Unit 8

WordPress

WordPress is a **popular open-source content management system (CMS)** used to create websites, blogs, and web applications without needing deep coding knowledge.

1. Free to use and modify under the GPL license.
2. Supported by a large developer community worldwide.
3. Backend is powered by **PHP** and **MySQL** database.
4. Allows users to **easily create, edit, and manage content** using a simple admin dashboard.
5. No need to write code for most tasks.
6. Offers thousands of **free and premium themes** for design customization.
7. Developers can build **custom themes and plugins** for advanced features.
8. Supports custom post types and page builders.
9. Generates clean URLs, mobile-responsive layouts, and supports SEO plugins.
10. Helps rank better in search engines.
11. Can turn into an online store using **WooCommerce plugin**.
12. Supports payment gateways, product management, inventory, etc.
13. Available in many languages.
14. Can assign roles like **admin, editor, author, contributor**.

Positive aspects and limitations of wordpress

Positive Aspects	Limitations
Easy to use and **user-friendly**	Can have **security vulnerabilities** if not managed well
Open-source and **free** to use	May suffer from **slower performance** with too many plugins
Extensibility through **plugins**	Requires **regular maintenance** and updates
Customizable design through themes and coding	May not be suitable for **complex applications**
Highly **SEO-friendly**	**Limited scalability** for enterprise-level websites
Large **support community** and resources	Needs knowledge of **coding** for advanced customization
Can easily convert to an **e-commerce site**	Possible **bloat** from unused themes/plugins

---